13. Probabilistic models 2

Machine learning

Department of Biomedical Engineering,
FEEC, Brno University of Technology

Probabilistic models - lectures outline

- Probability introduction + Bayes rule
- Probabilistic models - parameters estimation:
  - Maximum likelihood
  - Maximum a-posteriory
  - Bayesian inference

Probabilistic models 1

- Probabilistic models - example applications:
  - Naive Bayes classifier
  - Gaussian mixture model – EM algorithm
  - Logistic regression

Probabilistic models 2

Naive Bayes classifiers

Bayesian classifiers

Given measured data $\boldsymbol{x}$ and the class prediction $y \in \{y_1, \dots y_K\}$. We want to obtain the posterior probability of the class:

$$p(y_k|\boldsymbol{x})$$

This conditional probability for k-class can be derived from observed data.

This probability can be derived from observed data as frequency of samples with k-class.

We apply a Bayesian rule:

$$p(y_k|\boldsymbol{x}) = \frac{p(\boldsymbol{x}|y_k)p(y_k)}{p(\boldsymbol{x})} = \frac{p(x_1, x_2 \dots x_d|y_k)p(y_k)}{\sum_k p(x_1, x_2 \dots x_d|y_k)p(y_k)}$$

Sum over K discrete classes – it is possible.

and we are most intrested in most probable class:

$$y_k^* = argmax_{y_k} p(\boldsymbol{x}|y_k)p(y_k)$$

We can select from set $\boldsymbol{x}$ only samples with *k-th* class and fit parameters of probability distribution $p_\theta(x_{1,\dots}, x_d|y_k)$ by e.g. ML estimation.

However, this distribution is *d*-dimensional and therefore fitting can be difficult.

Naive Bayes classifiers

$$y_k^* = argmax_{y_k} p(x_{1,\dots}, x_d \,|y_k\,) \cdot p(y_k)$$

Assume that the features are mutually independent, conditional on given category y_k

$$y_k^* = argmax_{y_k} p(x_{1,\dots}, x_d \,|y_k\,) \cdot p(y_k) \;= argmax_{y_k} p(y_k) \cdot \prod_{i=1}^{d} p_\theta(x_i|y_k)$$

The task is to find distributions $p(x_i|y_k)$ described by parameter vector $\theta_{i,k}$ for *i-th* feature conditional by *k-th* class, thus we obtain $k \times d$ distributions.

The parameters of this distribution can be found with probability model based on observed data as follows.

We have observed set of *n* samples for *i-th* feature $[f_1, f_2, \dots, f_n]$. Then we estimate parameter $\boldsymbol{\theta}_{i,k}$ for set of samples $\boldsymbol{f}_{i,k}$ corresponding to k-class. We are fitting distribution only to samples corresponding to k-class, because $p(x_i|y_k)$ is conditioned by y_k.

$$\theta_{MAP}^* = argmax_{\theta_{i,k}} \, p(\theta_{i,k}|\boldsymbol{f}_{i,k}) = argmax_{\theta_{i,k}} \frac{p(\boldsymbol{f}_{i,k}|\theta_{i,k}) \cdot p(\theta_{i,k})}{p(\boldsymbol{f}_{i,k})} \qquad \boldsymbol{f}_{i,k} = \boldsymbol{f}(\boldsymbol{x}_i|y_k)$$

Naive Bayesian classifiers

- Naive – assume that features are mutually independent:
  - We get less complex model – easy to estimate, but…
  - in the real-world data it is not true – features are often somehow mutually dependent, but…
  - even the wrong and simple model can still perform better than the correct and complex model.

Pros:

- Robust to missing features – only existing features can be used in the product
- Robust to dependency of features
- Selection of the most appropriate distribution for each feature
- Parameters of distributions can be estimated by MLE/MAP – easy to compute and interpretable
- Because of conditional independency of features on given category we can estimate $\boldsymbol{\theta}$ more accurately with less data – product of 1D distributions instead of complex ND distribution

Cons:

- Must assume distributions of $\boldsymbol{\theta}$

Gaussian mixture model

Multivariate Gaussian function

d-dimensional Gaussian function:

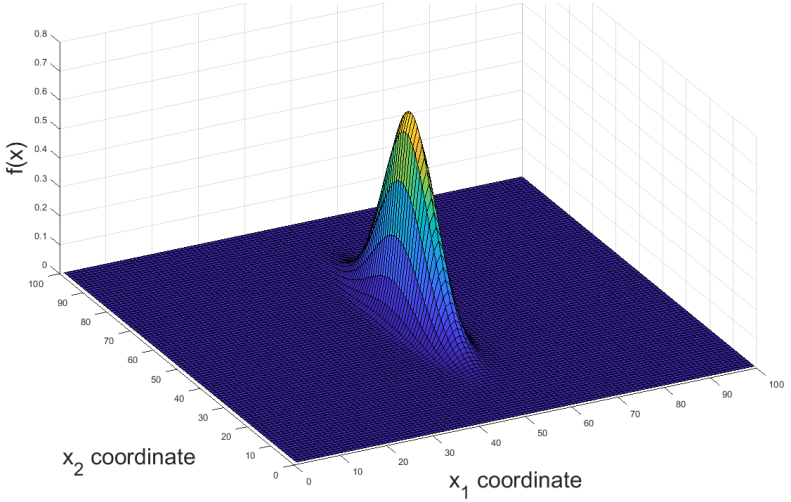
$$f(\mathbf{x}) = (2\pi)^{-\frac{\mathrm{d}}{2}} \ |\boldsymbol{\Sigma}|^{-\frac{1}{2}} \ \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^T\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)$$

$$\boldsymbol{\mu} = (\mu_1, \mu_2, \ldots, \mu_\mathrm{d})^\mathrm{T} \qquad \mathbf{x} = (\mathrm{x}_1, \mathrm{x}_2, \ldots, \mathrm{x}_\mathrm{d})^\mathrm{T}$$

$$\boldsymbol{\Sigma} = \begin{bmatrix} \sigma_1^2 & \cdots & \sigma_1\sigma_d \\ \vdots & \ddots & \vdots \\ \sigma_d\sigma_1 & \cdots & \sigma_d^2 \end{bmatrix}$$

for 2-dimensional case:

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \qquad \boldsymbol{\mu} = \begin{pmatrix} \mu_1 \\ \mu_2 \end{pmatrix} \qquad \boldsymbol{\Sigma} = \begin{bmatrix} \sigma_1^2 & \sigma_1\sigma_2 \\ \sigma_2\sigma_1 & \sigma_2^2 \end{bmatrix}$$



Estimation using MLE:

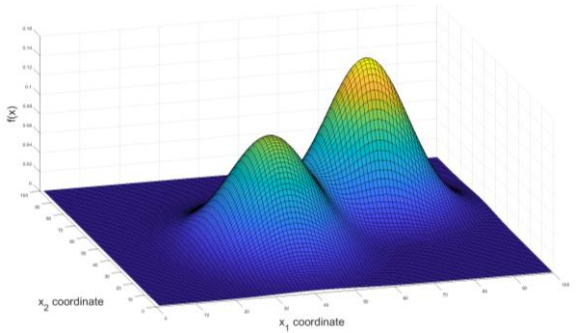
$$\boldsymbol{\mu}_{\mathbf{MLE}} = \frac{1}{N}\sum_{i=1}^{N} \boldsymbol{x_i} = \overline{\boldsymbol{x}}$$

$$\boldsymbol{\Sigma}_{\mathbf{MLE}} = \frac{1}{N}\sum_{i=1}^{N} (\boldsymbol{x_i} - \boldsymbol{\mu_{MLE}})^{\boldsymbol{T}} \, (\boldsymbol{x_i} - \boldsymbol{\mu_{MLE}})$$

Gaussian mixture model

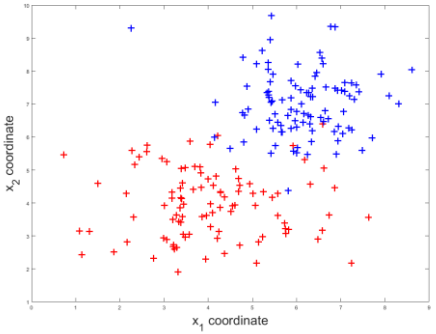
Mixture model

$$f(x; a_1, a_2, \dots, a_K, w_1, \dots, w_k) = \sum_{k=1}^{K} w_k p(x; a_k) \qquad \sum_{k=1}^{K} w_k = 1$$



Multivariate Gaussian mixture model:

$$f(\boldsymbol{x}; \boldsymbol{\mu_1}, \dots, \boldsymbol{\mu_K}, \boldsymbol{\Sigma_1}, \dots, \boldsymbol{\Sigma_K}, w_1, \dots, w_K) = \sum_{k=1}^{K} w_k \, p(\boldsymbol{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$



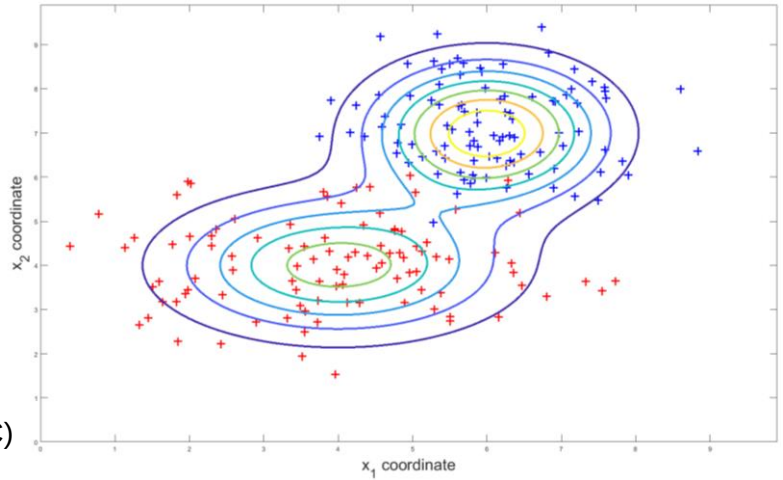
Gaussian mixture model estimation

Problem definition for K-modal case:

$$f(\boldsymbol{x}|\boldsymbol{\mu_1}, \dots, \boldsymbol{\mu_K}, \boldsymbol{\Sigma_1}, \dots, \boldsymbol{\Sigma_K}, w_1, \dots, w_K) = \sum_{k=1}^{K} w_k \, p(\boldsymbol{x}|\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

- to find the parameters (mean, variance and weights) of Mixture model

MLE or MAP cannot be used, becouse labels are unknown – we dont know which points should be used for which Gaussian



Therefore, the estimation must be solved via iterative algorithms (e.g. EM, MCMC)

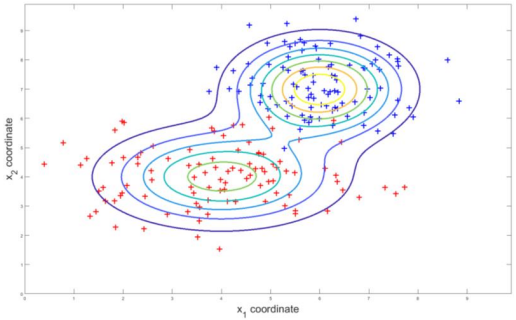
Gaussian mixture model estimation

1) Unsupervised learning method for **unlabeled** data clustering into **known number** of classes. („improved k-means“ with soft assignments and ellipsoidal clusters)

2) Approximation of unknown complex distribution by lot of Gaussians.

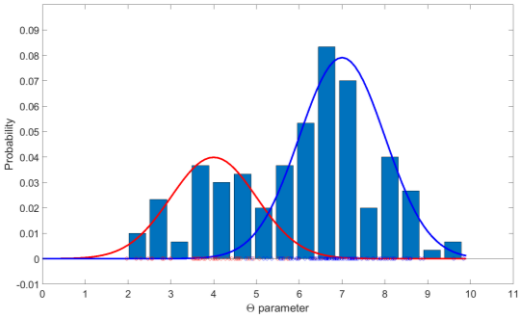
A probabilistic model that assumes all the data points are generated from a **mixture** of a finite number of Gaussian distributions with unknown parameters.

The aim is thus to **estimate parameters** of Gaussian distributions.



2D clustering into classes

1D fitting on distribution



Expectation – maximization (EM) - formally

Given the statistical model which generates observed data x and missing values z, likelihood for MLE is:

$$L(\theta; x) = p(x|\theta) = \int p(x, z|\theta)dz = \int p(z|x, \theta)p(x|\theta)dz$$

However, $p(z \mid x,\theta)$ cannot be calculated because z is unknown.

The EM algorithm seeks to find the MLE of the marginal likelihood by iteratively applying these two steps:

Expectation step – calculate expected value of the log likelihood of θ given current estimate θ^t

$$Q(\theta|\theta^t) = E_{z|x,\theta^t}\, log\, L(\theta; x, z)$$

Maximization step – find parameters that maximize this quantity:

$$\theta^{t+1} = argmax_\theta Q(\theta|\theta^t)$$

Expectation – maximization (EM)

1) Initialize parameters θ

2) Expectation (E) step – compute the probability of each possible value of z , given estimate of θ

3) Maximization (M) step – use estimate of z for better estimate of θ

4) Iterate between E and M step

Expectation – maximization (EM) for GMM

E – step – evaluate likelihood for each point x_i and each Gaussian k - $p(x_i; \theta)$ and normalize to get probability of this point to be from class k:

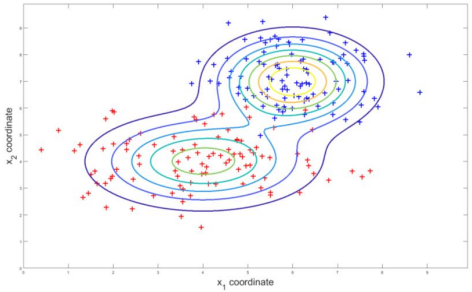
$$q(\boldsymbol{x_i}; \boldsymbol{\Theta_k}) = \frac{w_k p(\boldsymbol{x_i}; \boldsymbol{\Theta_k})}{\sum_{j=1}^{K} w_j \, p(\boldsymbol{x_i}; \boldsymbol{\Theta_j})}$$

$$p(\boldsymbol{x_i}; \boldsymbol{\Theta_k}) = (2\pi)^{-\frac{d}{2}} \; |\boldsymbol{\Sigma_k}|^{-\frac{1}{2}} \; exp\left(-\frac{1}{2}(\boldsymbol{x_i} - \boldsymbol{\mu_k})^T \boldsymbol{\Sigma_k^{-1}} (\boldsymbol{x_i} - \boldsymbol{\mu_k})\right)$$

$$\boldsymbol{\Theta_k} = [\boldsymbol{\mu_k}, \boldsymbol{\Sigma_k}]$$

M – step – get MLE estimate for each Gaussian – points are weighted by probability of points to be from class k:

$$w_k = \frac{1}{N} \sum_{i=1}^{N} q(\boldsymbol{x_i}; \boldsymbol{\Theta_k}), \qquad \mu_k = \frac{\sum_{i=1}^{N} q(\boldsymbol{x_i}; \boldsymbol{\Theta_k}) \boldsymbol{x_i}}{\sum_{i=1}^{N} q(\boldsymbol{x_i}; \boldsymbol{\Theta_k})}, \qquad \boldsymbol{\Sigma_k} = \frac{\sum_{i=1}^{N} q(\boldsymbol{x_i}; \boldsymbol{\Theta_k})(\boldsymbol{x_i} - \boldsymbol{\mu_k})(\boldsymbol{x_i} - \boldsymbol{\mu_k})^T}{\sum_{i=1}^{N} q(\boldsymbol{x_i}; \boldsymbol{\Theta_k})}$$



Gaussian mixture model estimation by EM

Algorithm for case of two 2D Gaussian (d=2, K=2)

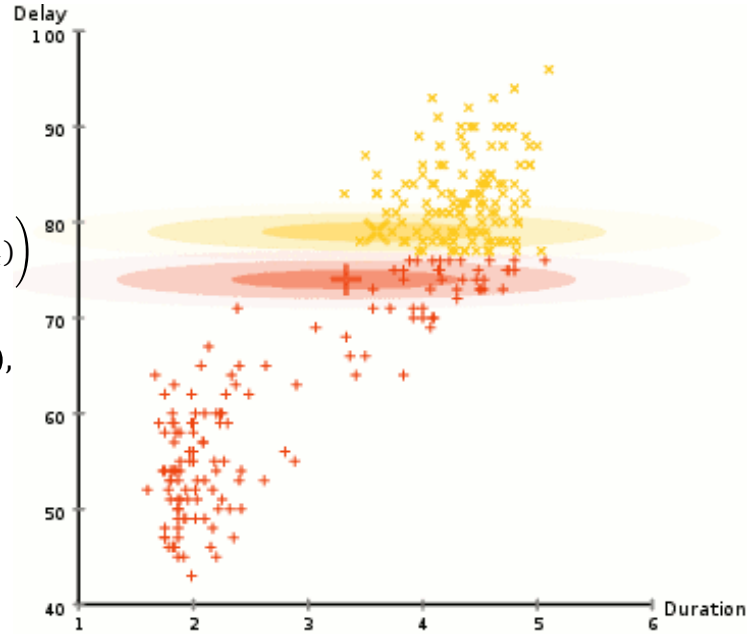
1. Initialization of $\boldsymbol{\mu_1}, \boldsymbol{\mu_2}, \boldsymbol{\Sigma_1}, \boldsymbol{\Sigma_2}, w_1, w_2$
2. calculate normalized likelihoods $q(\boldsymbol{x_i}; \boldsymbol{\Theta_1})$ and $q(\boldsymbol{x_i}; \boldsymbol{\Theta_2})$ for all points (E-step)

$$q(\boldsymbol{x_i}; \boldsymbol{\theta_k}) = \frac{w_k p(\boldsymbol{x_i}; \boldsymbol{\theta_k})}{\sum_{j=1}^{K} w_j \, p(\boldsymbol{x_i}; \boldsymbol{\theta_j})} \qquad p(\boldsymbol{x_i}; \boldsymbol{\theta_k}) = (2\pi)^{-\frac{d}{2}} \; |\boldsymbol{\Sigma_k}|^{-\frac{1}{2}} \; exp\left(-\frac{1}{2}(\boldsymbol{x_i} - \boldsymbol{\mu_k})^T \boldsymbol{\Sigma_k}^{-1} (\boldsymbol{x_i} - \boldsymbol{\mu_k})\right)$$

3. calculate $w_1 \, and \, w_2$ as mean of $q(\boldsymbol{x_i}; \boldsymbol{\theta_k})$ over all points $w_k = \frac{1}{N} \sum_{i=1}^{N} q(\boldsymbol{x_i}; \boldsymbol{\Theta_k})$, and re-estimate parameters $\boldsymbol{\mu_1}, \boldsymbol{\mu_2}, \boldsymbol{\Sigma_1}$ and $\boldsymbol{\Sigma_2}$ (M - step)

$$\mu_k = \frac{\sum_{i=1}^{N} q(\boldsymbol{x_i}; \boldsymbol{\Theta_k}) \boldsymbol{x_i}}{\sum_{i=1}^{N} q(\boldsymbol{x_i}; \boldsymbol{\Theta_k})}, \qquad \boldsymbol{\Sigma_k} = \frac{\sum_{i=1}^{N} q(\boldsymbol{x_i}; \boldsymbol{\Theta_k})(\boldsymbol{x_i} - \boldsymbol{\mu_k})(\boldsymbol{x_i} - \boldsymbol{\mu_k})^T}{\sum_{i=1}^{N} q(\boldsymbol{x_i}; \boldsymbol{\Theta_k})}$$

4. repeat step 2. - 3. until stopping criteria are not met



Stopping criterion:

- Maximal number of iterations
- Change of parameters is under limit $\left|\mu^{i+1} - \mu^{i}\right| < \epsilon$

Logistic regression

Logistic regression

Logistic regression is a method for creating the statistical model that is used in classification problems. It allows us to take some features and predict the correct class, or better - the probability of that class.

Linear *versus* logistic regression:

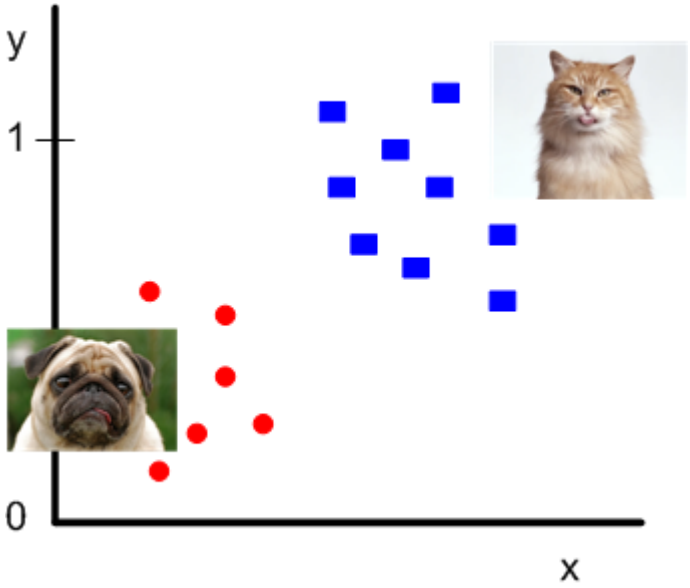
- we can predict if student pass the exam (yes/no) based on number of hours spent by studying. That is task for logistic regression.
- we can predict how many points the student get (“almost” continuous variable) based on number of hours spent by studying. That is task for linear regression.

Definition

We want to predict what is the probability that we classify CAT (i.e. y_i=1), based on given input (vector of features $\mathbf{x_i}$) and parameters of the model **w**. Written in math:

$$p(y_i = 1|\boldsymbol{x}_i, \boldsymbol{w}) = \ ?$$

It doesn't make sense to use the linear model, because it will just fit the line through our data. But we can use some function to transform this linear model into something more convenient.



Sigmoid function

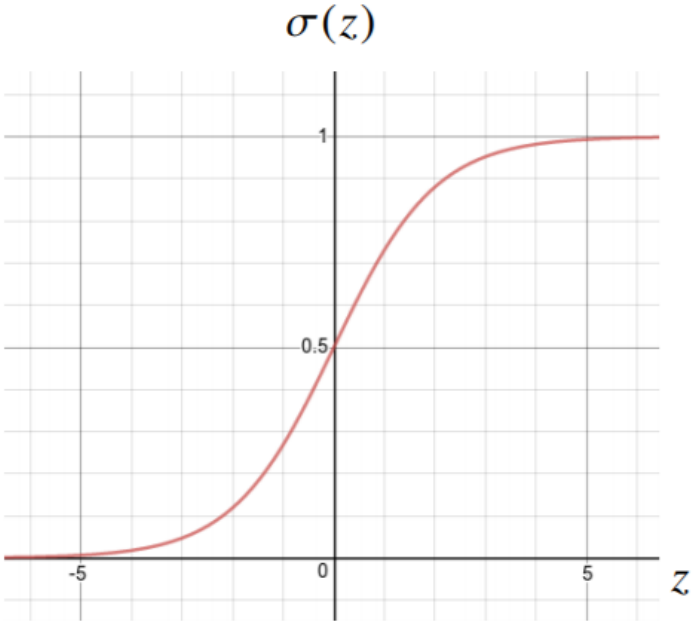
For this purpose, we will use the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The useful properties of this function are:

- whatever the input is, it is transformed between 0 and 1.
- the derivative of a sigmoid with respect to input is very nice/useful (see later):

$$\frac{d\,\sigma(z)}{d\,z} = \sigma(z) \cdot (1 - \sigma(z))$$



Logistic regression

Now, we will use this sigmoid function to transform our linear model:

$$\theta = \sigma(\boldsymbol{w^T x}) = \frac{1}{1 + e^{-\boldsymbol{w^T x}}}$$

Bernoulli probability distribution: $f(x|\theta) = \theta^x (1-\theta)^{1-x}$

We can now look on our data in a different way: We can interpret labels as a Bernoulli random variable given the model described by sigma function.

Now, we define likelihood as a “probability” that observed data x come from specific model (Bernoulli) with parameters $\boldsymbol{w}$. This likelihood function for my independent and identically distributed random variable has the following form:

$$\mathcal{L}(\boldsymbol{w}|\boldsymbol{y}, \boldsymbol{x}_1, \ldots, \boldsymbol{x}_n) = \prod_i^n \sigma(\boldsymbol{w^T x_i})^{y_i} \left(1 - \sigma(\boldsymbol{w^T x_i})\right)^{1-y_i}$$

Solution

It turns out that maximizing likelihood function can be difficult due to multiplications. Better way is to maximize Log of likelihood function:

$$\mathcal{L}_{log} = \log \mathcal{L}(\boldsymbol{w}|, \boldsymbol{y}, \boldsymbol{x_1}, \dots, \boldsymbol{x_n}) = \sum_{\boldsymbol{i}} y_i \log(\sigma(\boldsymbol{w^T x}_i)) + (1 - y_i) \log(1 - \sigma(\boldsymbol{w^T x_i}))$$

We have to maximize this function with respect to w. So, we must take the derivative of $\mathcal{L}_{log}$

$$\frac{d\, \mathcal{L}_{log}(\boldsymbol{w}|, \boldsymbol{y}, \boldsymbol{x_1}, \dots, \boldsymbol{x_n})}{dw^{(j)}} = \sum_{\boldsymbol{i}} \left(\boldsymbol{y_i} - \sigma(\boldsymbol{w^T x}_i)\right) x_i^{(j)}$$

And finally, we can write the gradient ascent optimization:

$$w_{new}^{(j)} = w_{old}^{(j)} + \mu \sum_{i} \left(y_i - \sigma(\boldsymbol{w^T x_i})\right) x_i^{(j)}$$

Probabilistic models - lectures outline

- Probability introduction + Bayes rule
- Probabilistic models - parameters estimation:
  - Maximum likelihood
  - Maximum a-posteriory
  - Bayesian inference

Probabilistic models 1

- Probabilistic models - example applications:
  - Naive Bayes classifier
  - Gaussian mixture model – EM algorithm
  - Logistic regression

Probabilistic models 2